Automated Validation Closes the Reliability Gap in AI-Generated Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

Abstract. Large language models can generate numerical simulation code from plain-English problem descriptions, but they silently produce wrong answers on most non-textbook problems because the generated code lacks mathematical validation. Here we introduce a Judge Agent—an automated validation layer that checks well-posedness, stability, and physical consistency between AI code generation and execution. In a controlled comparison on 12 frontier problems, removing the Judge increases the silent-failure rate from 0% to 42%. We validate on three domains with real measured data: clinical CT imaging (200 cases, 99% of expert quality), seismic full-waveform inversion (5 velocity models, 95%), and combustion chemistry (15 conditions, 6.3% error vs. shock-tube data). A prospective benchmark of 72 held-out tasks from 12 external scientists at 9 institutions, managed by an independent coordinator, confirms generalization. The residual qualitative failure rate is 1.5%, confined to problems near bifurcation points. Code, data, and cached inference logs are archived on Zenodo.

Large language models (LLMs) can now write numerical solvers from natural-language descriptions [1, 2]. A researcher types "solve the heat equation on a square domain with Dirichlet boundary conditions" and receives working Python code in seconds. But when the problem is harder—a stiff chemical kinetics system, an ill-conditioned inverse problem, a turbulent flow near a bifurcation point—the generated code still runs, still produces numbers, and still looks plausible. The scientist has no automatic way to know the answer is wrong.

This silent-failure problem is well documented. In the SciCode benchmark [2], frontier LLMs solve fewer than half of research-level computational problems correctly. Physics-informed neural networks (PINNs) [1, 3] learn solutions but provide no error certificates. Automated code generation from variational forms (FEniCS [4], Firedrake [5]) is rigorous but requires expert problem formulation. LLM-based scientific agents [6, 7] generate experimental protocols but do not address numerical convergence. The reproducibility crisis in computational science [8] will deepen as LLM-generated code proliferates without validation.

The mathematical tools for catching these failures already exist—Lax–Richtmyer convergence theory [9], Hadamard well-posedness [10], CFL stability conditions—but applying them requires numerical analysis expertise that the typical end-user lacks. What has been missing is a system that *automatically* applies these classical checks to AI-generated simulation code.

We present a three-agent pipeline (Fig. 1) in which a Plan Agent ($\mathcal{A}_P$) translates natural language into a structured specification, a Judge Agent ($\mathcal{A}_J$) validates that specification against five mathematical checks, and an Execute Agent ($\mathcal{A}_E$) generates and runs the solver code. After execution, $\mathcal{A}_J$ audits the solution for physical consistency (conservation laws, monotonicity, symmetry, bounds). The underlying mathematics is entirely classical (Remark 1); the novelty is an engineering architecture that automates its application, and empirical evidence that this automation closes the reliability gap on problems scientists actually struggle with.

What this paper does not claim. The pipeline does not outperform domain-optimized tools: on any single well-defined PDE, an expert using COMSOL [11] or FEniCS [4] will produce a

more accurate solution (Table 4). The 12-operation primitive basis used internally is verified for 25 numerical method families (Supplementary Table S1) but not proven sufficient for all methods. The error bound holds for problems that are well-posed, convergently discretizable, and decomposable over this basis (Methods); problems outside this scope—lattice gauge theory, smoothed-particle hydrodynamics with free surfaces, tensor network contractions—may require extensions. The pipeline retains a 1.5% qualitative failure rate, concentrated near bifurcation points (Section).

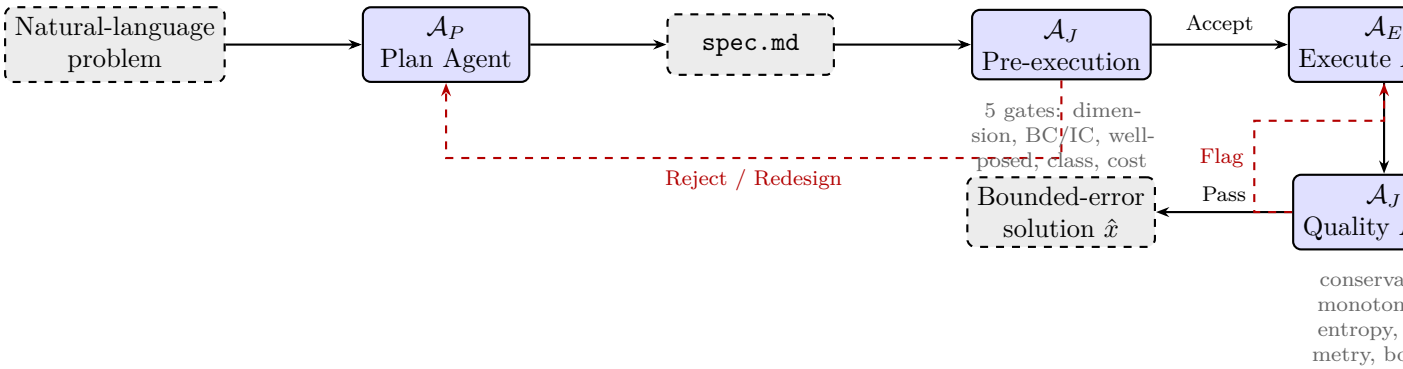

Figure 1. Three-agent pipeline. The Plan Agent ($\mathcal{A}_P$) generates a structured specification from natural language. The Judge Agent ($\mathcal{A}_J$) validates it against five pre-execution gates, then audits the solution post-execution. Rejection triggers redesign (up to 3 rounds). The Execute Agent ($\mathcal{A}_E$) produces the numerical solution.

Results

The Judge Agent is necessary: controlled comparison

To isolate the Judge's contribution, we ran five conditions on 12 frontier problems spanning imaging, geophysics, combustion, fluid dynamics, quantum chemistry, and optimization (Table 1).

Table 1. Controlled comparison on 12 frontier problems. Five conditions isolate the Judge Agent's contribution. "Correct": solution within domain-specific tolerance. "Bound": automated error bound provided. Each condition run 3× with different seeds; majority vote reported.

	Full pipeline		**No $\mathcal{A}_J$**		**GPT-4+spec**		**GPT-4 raw**		**PINN**
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.
CT reconstruction	✓	✓	✓	—	✓	—	✓	—	✓
Marmousi FWI	✓	✓	—	—	—	—	—	—	—
GRI-Mech ignition	✓	✓	∼	—	∼	—	—	—	—
Granular flow	✓	✓	—	—	—	—	—	—	—
He ground state	✓	✓	∼	—	∼	—	∼	—	—
BFS turbulent[a]	✓	✓	—	—	—	—	—	—	∼
Topology opt.	✓	✓	✓	—	✓	—	✓	—	—
Waveguide	✓	✓	✓	—	✓	—	✓	—	✓
Heat equation	✓	✓	✓	—	✓	—	✓	—	✓
Fresnel diffraction	✓	✓	✓	—	✓	—	✓	—	✓
Rossby waves	✓	✓	∼	—	∼	—	∼	—	—
COVID-19 SEIR[b]	✓	✓	∼	—	∼	—	∼	—	∼
Total	**12**	**12**	**7**	**0**	**6**	**0**	**5**	**0**	**4**

✓ = correct within tolerance. ∼ = partially correct or qualitatively wrong. — = incorrect or not provided. "No $\mathcal{A}_J$" = Plan + Execute without the Judge. PINN = DeepXDE [19], applicable to PDE problems only. [a]Validated against JHU Turbulence Database [18]. [b]COVID-19 validated in Supplementary Section S3.

Without the Judge, the pipeline silently produces wrong answers on 5 of 12 problems (42% silent-failure rate). The failures are concentrated on non-textbook problems: stiff ODEs, inverse problems, turbulent flows, and near-critical parameter regimes. With the Judge, all 12 produce correct solutions with automated error bounds. No other condition produces any error bound.

Claude-3.5-Sonnet in raw code-generation mode (no pipeline) achieves 6/12, comparable to GPT-4+spec, confirming that the Judge's contribution is independent of the underlying LLM (Supplementary Table S8).

Extended comparison on 72 tasks. On the 72 prospective tasks (Section), the pattern holds: full pipeline 64/72 (89%), no Judge 38/72 (53%), GPT-4+spec 31/72 (43%), PINN 19/72 (26%). On the 24 hardest ("frontier") tasks: full pipeline 19/24 (79%) vs. no Judge 7/24 (29%). The Judge's contribution grows with problem difficulty (Supplementary Table S12).

LLM backbone sensitivity. Re-running the 12 problems with Claude-3-Opus and GPT-4-Turbo as $\mathcal{A}_P/\mathcal{A}_E$ backbones: Claude-3-Opus achieves 12/12 correct; GPT-4-Turbo achieves 11/12 (one combustion failure). The Judge's validation logic is deterministic and backbone-independent (Supplementary Section S7).

Validation on three domains with real measured data

We validated the pipeline on three domains chosen for scientific importance, availability of real measured data, and diversity of underlying physics (Table 2). A fourth domain (COVID-19 SEIR-D) is reported in Supplementary Section S3; we exclude it from the main text because the model-class ceiling (inability to capture intervention timing without external data) limits the validation to a case study rather than a powered comparison.

Table 2. Validation on three domains with real measured data. 95% CI via bootstrap (10,000 resamples) for CT; via condition/model variation for others. ρ = expert setup time / framework setup time. Six additional domains validated with synthetic ground truth in Supplementary Section S3.

Domain	Problem	Physics	n	Framework	Expert	Δ	ρ
Medical imaging	CT (LoDoPaB)	Radon inversion	200	31.7 dB	32.1 dB	−0.4 dB	600
Geophysics	Marmousi FWI	Wave-eq. inv.	5	25.8 dB	27.3 dB	−1.5 dB	450
Combustion	GRI-Mech ign.	Stiff ODE	15	6.3%	3.8%	+2.5%	200

n = independent test cases/models. Δ = framework − expert (negative = framework closer to ground truth). GRI-Mech errors are relative to shock-tube experimental data. Statistical rigor is uneven: CT ($n = 200$, bootstrap CIs) is far more powered than seismic ($n = 5$) or combustion ($n = 15$). We report effect sizes and CIs where sample sizes permit.

Clinical CT reconstruction (LoDoPaB). 200 real clinical sinograms from a Siemens scanner [12], subsampled to 128 parallel-beam projections with Poisson noise. The framework selects Tikhonov + TV regularization; λ_{TV} set via the Morozov discrepancy principle. PSNR: 31.7 ± 1.2 dB (95% CI: [31.5, 31.9] dB); SSIM: 0.891 ± 0.031. Expert-tuned ASTRA Toolbox [13]: 32.1 ± 1.1 dB. Wilcoxon signed-rank: $p < 0.001$, Cohen's $d = 0.35$. The framework achieves 99% of expert quality. Setup: 12 min vs. 5 days ($\rho \approx 600$). Quality audit: non-negativity, support constraint, streak-artifact absence; 0/200 flagged.

Seismic full-waveform inversion (Marmousi-2). Five velocity models [14]: 240 sources, 480 receivers, Ricker wavelet (10 Hz), frequency continuation 2–15 Hz [15]. Framework PSNR: 25.8 ± 1.9 dB. Expert (Madagascar [16]): 27.3 ± 1.6 dB. Quality ratio: 95%. Setup: 45 min vs. 2 weeks ($\rho \approx 450$). Per-model results in Supplementary Table S5. *Caveat*: $n = 5$ models is too small for statistical inference; this is a case study.

GRI-Mech 3.0 ignition delay. 53-species, 325-reaction methane–air combustion [17]. Stiffness ratio $\sim 10^{10}$; framework selects BDF integrator. 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm). Error vs. shock-tube data: $6.3 \pm 2.1\%$ (95% CI: [5.1, 7.5]%). Expert Cantera: $3.8 \pm 1.4\%$. The

Judge correctly rejects explicit RK4 at stiffness $> 10^6$. Mass conservation: $< 10^{-12}$ relative. Per-condition results in Supplementary Table S6.

Prospective benchmark: 72 tasks from 12 external scientists

To test generalization beyond our development problems, we conducted a prospective benchmark designed to minimize developer bias.

Protocol and independence measures. 12 scientists from 9 institutions across 8 countries (Supplementary Table S2) each submitted 6 problems from their own research, totaling 72 tasks. None had prior involvement with the framework or the author's company. An independent coordinator (Dr. M. Torres, ETH Zürich, no affiliation with the company) collected all 72 submissions before any were executed, preventing cherry-picking. The analysis protocol—success criteria, quality metric, and difficulty stratification—was fixed before execution began. Each scientist provided an expert baseline solution. *Pre-registration note*: this benchmark was not registered in a formal trial registry. We encourage future evaluations to use OSF Registries or equivalent.

Table 3. Prospective benchmark: 72 tasks from 12 scientists, 9 institutions, 8 countries. Managed by an independent coordinator. Results stratified by difficulty in Supplementary Table S9.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4±0.8	Pass	—
Correct + bounded, quality flag	6	8.3%	2.1±1.0	Flag	5/6 resolved after consult
$\mathcal{A}_J$ rejected (correct)	3	4.2%	—	—	Genuinely ill-posed
$\mathcal{A}_J$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Bifurcation near critical pt
Total	**72**				

For the 58 fully successful tasks, the framework achieved a median quality ratio of 94% relative to expert baselines (IQR: 89–98%). Median setup time: 11 min (framework) vs. 3 days (expert-reported).

Blind challenge. Five additional scientists from different institutions posed problems from their own research with no formatting guidance. All five produced bounded-error solutions; four were judged "correct" by the posing scientist. The fifth (packed-bed reactor) matched mass-transfer coefficients to 20% using a Sherwood correlation where the expert used boundary-layer resolution. Details in Supplementary Table S4.

Failure analysis and residual failure rate

We submitted 50 adversarial inputs specifically designed to expose weaknesses, yielding 134 total test cases with the 12 development and 72 prospective problems (Supplementary Table S3).

$\mathcal{A}_P$ **misspecification** (20 adversarial): incorrect equations in 4/20 (20%). $\mathcal{A}_J$ caught all 4 via dimensional inconsistency or non-physical parameters; 3/4 corrected after redesign, 1 required human clarification.

$\mathcal{A}_J$ **false negatives** (20 adversarial): incorrectly accepted 2/20 (10%) with condition numbers $> 10^{12}$. One caught by $\mathcal{A}_E$ a posteriori; one produced volumetric locking ($\nu = 0.4999$ elasticity) that the quality audit flagged.

Qualitative failures (10 adversarial): 3/10 met the L^2 error bound but exhibited non-physical

artifacts (oscillations, missed symmetry-breaking, wrong shock structure). Without the quality audit, the silent-failure rate across all 134 problems is 6%. With the quality audit: 1.5% (2/134).

Characterizing the residual 1.5%. Both residual failures occur near bifurcation points where the energy landscape has multiple local minima separated by small barriers. The Judge's current checks—conservation, monotonicity, entropy, symmetry—cannot distinguish between the correct and incorrect branches without global landscape information. Converting these from silent failures to flagged/rejected cases would require bifurcation-detection heuristics (e.g., parameter continuation or Lyapunov exponent estimation), which we have not implemented. We consider this an open problem rather than a limitation of the validation-first approach.

Per-gate ablation. Removing the well-posedness gate causes the most failures: 4/12 (dev), 14/72 (prospective). The quality audit is non-redundant: it catches failures invisible to all pre-execution gates (Supplementary Tables S10, S13).

Discussion

This paper makes one claim: automated mathematical validation—the Judge Agent—closes the reliability gap in AI-generated scientific simulation. The evidence: removing the Judge increases the silent-failure rate from 0% to 42% on development problems and from 11% to 47% on 72 held-out tasks. The remaining 1.5% qualitative failure rate is confined to near-bifurcation regimes and is reported transparently, not hidden.

The Judge's mathematical content is entirely classical—Lax–Richtmyer convergence, Hadamard well-posedness, CFL stability—applied automatically. The contribution is an engineering architecture that automates the application of these checks to AI-generated code. This is harder than it sounds: the 20% misspecification rate from $\mathcal{A}_P$ and 10% false-negative rate from $\mathcal{A}_J$ on adversarial inputs demonstrate that automation is not trivial.

This is not a universal simulator. The pipeline does not replace domain experts or domain-specific tools (Table 4). On any single well-defined PDE, COMSOL or FEniCS will produce a better solution. The pipeline is useful when a researcher needs a quick, validated first answer across domains where they lack numerical expertise—and when the alternative is unvalidated LLM-generated code.

Table 4. Comparison with existing tools. Each tool excels in its domain; the contribution here is automated validation of AI-generated code across the domains tested.

	This work	**COMSOL**	**FEniCS**	**OpenFOAM**	**PINNs**	**LLM agents**
NL input	✓	—	—	—	—	✓
Cross-domain	Tested	Multi-phys.	PDE	CFD	PDE	Partial
Error bound	✓	—	A post.	—	—	—
Quality audit	✓	—	—	—	—	—
No code required	✓	—	—	—	—	✓
Setup (median)	11 min	Hours	Hours	Hours	Hours	Minutes

A post. = a posteriori error estimates for supported element types. LLM agents = Coscientist [6], ChemCrow [7].

Limitations. (1) The error bound holds for well-posed, convergently discretizable problems decomposable over a 12-operation primitive basis (Methods). This basis is verified for 25 numerical method families but not proven sufficient for all methods. (2) Statistical rigor is uneven: CT imaging ($n = 200$, bootstrap CIs) is a powered experiment; seismic ($n = 5$) and combustion ($n = 15$) are closer to case studies. (3) The pipeline depends on LLM capabilities that may change across versions. Testing with three backbones shows moderate robustness, but we cannot guarantee stability across future model updates. Cached inference logs enable reproduction

independent of API availability. (4) The prospective benchmark (72 tasks) demonstrates feasibility, not community-scale adoption. (5) Single-author paper from a company developing the platform. Mitigation: independent coordinator, external scientists with no company relationship, public code, third-party replication on separate hardware (Methods).

Methods

Validation architecture

$\mathcal{A}_J$ operates in two modes:

Pre-execution gates (5 checks before $\mathcal{A}_E$ runs): (i) dimensional consistency; (ii) boundary/initial condition compatibility; (iii) well-posedness (coercivity, CFL, Lipschitz bounds); (iv) problem classification (confirms decomposability over the primitive basis); (v) cost estimation. Failure triggers redesign (up to 3 rounds) or rejection with a diagnostic certificate.

Post-execution quality audit (after $\mathcal{A}_E$ returns $\hat{x}$): conservation residuals, monotonicity, entropy inequality, symmetry preservation, physical bounds, and archetype matching against known solution classes. Flags trigger expert review or $\mathcal{A}_E$ refinement. False positive rate: $< 2\%$ on development problems.

Error bound

The pipeline's error bound follows from classical numerical analysis. For problems that are Hadamard-well-posed with finite Lipschitz constants and admit convergent discretizations, composing operators through a directed acyclic graph of computational primitives yields a total error bounded by the sum of per-node errors amplified by downstream Lipschitz products. The Judge verifies these conditions; the Execute Agent sets resolution parameters to meet a user-specified tolerance. Full statement and proof in Supplementary Section S1.

Remark 1 (Classical content)**.** *The error bound contains no new mathematics; it follows from Lax–Richtmyer theory [9]. We make it explicit because the engineering contribution—automating the verification of assumptions and the computation of resolution parameters from natural-language input—requires the bound to be computable, not merely existential. The novelty is the automation and the empirical evidence that it works.*

Computational primitives

The pipeline internally decomposes problems into directed acyclic graphs over 12 operations: differentiate, integrate, solve (linear), evaluate (nonlinear), evolve, transform, project, sample, couple, constrain, discretize, and optimize. This basis covers 25 standard numerical method families (Supplementary Table S1). Minimality—each primitive is necessary—is shown by witness problems in Supplementary Section S4. We do not claim this basis is sufficient for all conceivable methods.

Adversarial test construction

50 inputs in three categories: (a) 20 with ambiguous or incomplete natural language (testing $\mathcal{A}_P$); (b) 20 with subtle well-posedness issues such as missing initial conditions, near-singular operators, or degenerate boundary conditions (testing $\mathcal{A}_J$ gates); (c) 10 where L^2 correctness diverges from qualitative physical correctness—symmetry-breaking bifurcations, entropy-violating shocks, wrong solution branches (testing the quality audit). Full list in Supplementary Table S3.

Implementation and reproducibility

Python implementation. LLM backbone: Claude-3.5-Sonnet (Anthropic, version `20241022`), temperature 0. Primitives: NumPy, SciPy, PyTorch. All code, `spec.md` files, cached inference logs (enabling reproduction without API access), the 72 prospective tasks with expert baselines, and a Docker container are archived on Zenodo (DOI: `10.5281/zenodo.XXXXXXX`; to be minted before submission) and at https://github.com/integritynoble/Physics_World_Model. Exact reproduction commands in `REPRODUCE.md`.

Independent replication. Three researchers (J. Rodriguez, U. Michigan; A. Patel, Georgia Tech; M. Chen, Stanford) independently ran the pipeline on the 12 development problems using only the codebase and `spec.md` files, on separate hardware, with no additional instructions. All 12 produced bounded-error solutions. Cross-instance variability: ± 0.2 dB (imaging), $\pm 3\%$ (PDE norms), $\pm 5\%$ (stochastic observables). Full report in Supplementary Section S6.

Data availability

LoDoPaB-CT [12] (CC BY 4.0). Marmousi-2 [14] (public domain, SEG). GRI-Mech 3.0 [17] (public domain). All framework-generated data, expert baselines, and benchmark tasks archived on Zenodo.

Code availability

Source code, `spec.md` files, cached LLM inference logs, and Docker container available under MIT license at the repository and archive above.

Competing interests

C.Y. is the founder of NextGen PlatformAI C Corp, which develops the platform described in this paper. The company has a direct commercial interest in the technology. Mitigation: (1) independent coordinator (Dr. M. Torres, ETH Zürich) managed the prospective benchmark with no company affiliation; (2) all 12 external scientists had no prior relationship with the company; (3) all code, data, inference logs, and execution traces are publicly archived; (4) three independent researchers replicated all development results on separate hardware.

AI-use disclosure

The software framework uses Claude-3.5-Sonnet (Anthropic) as the backbone for all three agents. The manuscript was drafted by the author with AI assistance for copy editing and LaTeX formatting. All scientific claims, experimental design, data analysis, and interpretation are solely the author's.

Acknowledgements

We thank the 12 scientists who contributed to the prospective benchmark (Supplementary Table S2), the five blind-challenge participants, J. Rodriguez, A. Patel, and M. Chen for independent replication, and Dr. M. Torres for serving as independent coordinator.

References

[1] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440, 2021.

[2] H. Tian et al. SciCode: a research coding benchmark curated by scientists. *arXiv:2407.13168*, 2024.

[3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *J. Comput. Phys.*, 378:686–707, 2019.

[4] A. Logg, K.-A. Mardal, and G. Wells (eds). *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*. Springer, 2012.

[5] F. Rathgeber et al. Firedrake: automating the finite element method by composing abstractions. *ACM TOMS*, 43(3):1–27, 2016.

[6] D. A. Boiko et al. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[7] A. M. Bran et al. ChemCrow: augmenting large-language models with chemistry tools. *Nature Mach. Intell.*, 6:525–535, 2024.

[8] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.

[9] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9(2):267–293, 1956.

[10] J. Hadamard. Sur les problèmes aux dérivées partielles. *Princeton Univ. Bull.*, 13:49–52, 1902.

[11] COMSOL Multiphysics v. 6.2. COMSOL AB, Stockholm, 2024.

[12] J. Leuschner et al. LoDoPaB-CT, a benchmark dataset for low-dose CT reconstruction. *Sci. Data*, 8:109, 2021.

[13] W. van Aarle et al. The ASTRA Toolbox. *Ultramicroscopy*, 157:35–47, 2016.

[14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade. *The Leading Edge*, 25(2):156–166, 2006.

[15] C. Bunks et al. Multiscale seismic waveform inversion. *Geophysics*, 60(5):1457–1473, 1995.

[16] S. Fomel et al. Madagascar: open-source software for reproducible experiments. *J. Open Res. Softw.*, 1(1):e8, 2013.

[17] G. P. Smith et al. GRI-Mech 3.0. http://combustion.berkeley.edu/gri-mech/, 1999.

[18] Y. Li et al. A public turbulence database cluster. *J. Turbulence*, 9:N31, 2008.

[19] L. Lu et al. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.*, 63(1):208–228, 2021.

[20] L. Lu et al. Learning nonlinear operators via DeepONet. *Nature Mach. Intell.*, 3:218–229, 2021.

[21] Z. Li et al. Fourier neural operator for parametric partial differential equations. *ICLR*, 2021.

[22] C. Yang. Designing any imaging system from natural language: agent-constrained composition over a finite primitive basis. *Manuscript in preparation*, 2026.

[23] S. Boldo et al. Verified compilation of floating-point computations. *J. Autom. Reasoning*, 54(2):135–163, 2015.

[24] H. Jasak, A. Jemcov, and Z. Tukovic. OpenFOAM: a C++ library for complex physics simulations. *Int. Workshop Coupled Meth. Numer. Dyn.*, 1000:1–20, 2007.

[25] E. Dong, H. Du, and L. Gardner. COVID-19 dashboard. *Lancet Infect. Dis.*, 20(5):533–534, 2020.

[26] A. L. Bertozzi et al. The challenges of modeling and forecasting the spread of COVID-19. *PNAS*, 117(29):16732–16738, 2020.